

ADAMAS UNIVERSITY

Department of Computer Science & Engineering

SUBJECT: Software Engineering
COURSE CODE: CSE11117

Assignment: Online Food Ordering System

Software Requirements Specification (SRS) & System Architecture

---

Name:

Debopriya Dey

Academic Year:

2025–2026

November 1, 2026

ABSTRACT

The Online Food Ordering System is a web-based application designed to simplify the process of ordering food from restaurants. Traditional ordering through telephone calls or in-person visits can lead to delays, incorrect orders, limited menu visibility and difficulty tracking order progress. The proposed system provides a single platform where customers can create accounts, browse food items, add products to a cart, place orders, select a payment method and monitor order status.

The application supports three main operational roles: customers, restaurant administrators and delivery personnel. Customers can view categorized menus, manage cart quantities, confirm delivery details and review previous orders. Administrators can maintain menu information, update item availability, receive incoming orders and change preparation status. Delivery personnel can view assigned deliveries and update delivery progress.

The proposed implementation can use a responsive frontend built with HTML, CSS and JavaScript (or React), a Node.js/Express backend and a database such as MongoDB. Payment processing may be integrated through a secure payment gateway; for an academic prototype, payment can also be represented through a simulated transaction or cash-on-delivery option. This SRS defines the scope, interfaces, functional requirements, data model, process flows and quality requirements of the system.

Keywords: Online ordering, food menu, shopping cart, order management, payment, delivery tracking, SRS.

CONTENTS

Section	Page
Revision History	4
1 Introduction	5
2 Overall Description	7
3 External Interface Requirements	12
4 System Features	14
5 Other Nonfunctional Requirements	18
6 Other Requirements	20
A Glossary	23
B Analysis Models	24
C To Be Determined (TBD) List	24

Revision History

Version	Date	Primary Reason for Change	Author
1.0	10/09/2025	Initial Draft: Project scope, problem statement, and core food-ordering requirements gathering.	Debopriya Dey
1.1	22/09/2025	Added DFD Levels 0, 1 and 2, conceptual ER model, and three-tier architecture.	Debopriya Dey
2.0	05/10/2025	Integrated system features: authentication, menu browsing, cart, order placement, payment and tracking.	Debopriya Dey
2.1	01/11/2026	Final revision: aligned SRS structure, interface mockup and architecture with the Online Food Ordering System.	Debopriya Dey

Table 1: Document Revision History

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements of the Online Food Ordering System. It serves as a reference for developers, testers, project evaluators, restaurant administrators and end users. It explains what the system is expected to do and the constraints under which it should operate.

1.2 Document Conventions

- Functional requirements are identified as FR-X.Y, where X represents the feature number and Y represents the requirement sequence.
- Priority is classified as High (essential), Medium (important) or Low (optional enhancement).
- The terms “shall” and “must” indicate mandatory system behaviour; “may” indicates an optional capability.

1.3 Intended Audience and Reading Suggestions

- Developers: review Sections 2, 3, 4 and 6 for architecture, interfaces and implementation requirements.
- Testers: use Sections 4 and 5 to prepare functional and quality-assurance test cases.
- Restaurant administrators: review menu, order and availability management requirements.
- Customers: review the product functions and user-interface description for the ordering workflow.

1.4 Product Scope

The Online Food Ordering System provides an integrated digital channel for customers to order food from a restaurant or food-service provider. The system aims to reduce manual order-taking, improve order accuracy, make menu information easily accessible and provide visibility into order progress.

Core objectives

- Allow customers to register, log in and manage basic profile and delivery details.
- Display food items with names, descriptions, prices, categories and availability.
- Support cart operations, order confirmation, bill calculation and order history.
- Allow administrators to manage menu items and process incoming orders.
- Provide order-status updates from confirmation through preparation and delivery.

Out of scope for the initial academic prototype: multi-restaurant marketplace settlement, advanced route optimisation, loyalty programmes, live GPS navigation and production-grade payment settlement unless separately integrated.

1.5 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- Pressman, R. S. & Maxim, B. R., Software Engineering: A Practitioner’s Approach, 9th Edition.
- MongoDB Documentation, Database and CRUD Operations.
- Node.js and Express.js Documentation; React Documentation (if React is used).

2 Overall Description

2.1 Product Perspective

The system is a web application with a presentation layer, application-logic layer and data-persistence layer. The browser provides the customer and administrator interfaces. The backend validates requests, calculates order totals, manages order state and communicates with the database.

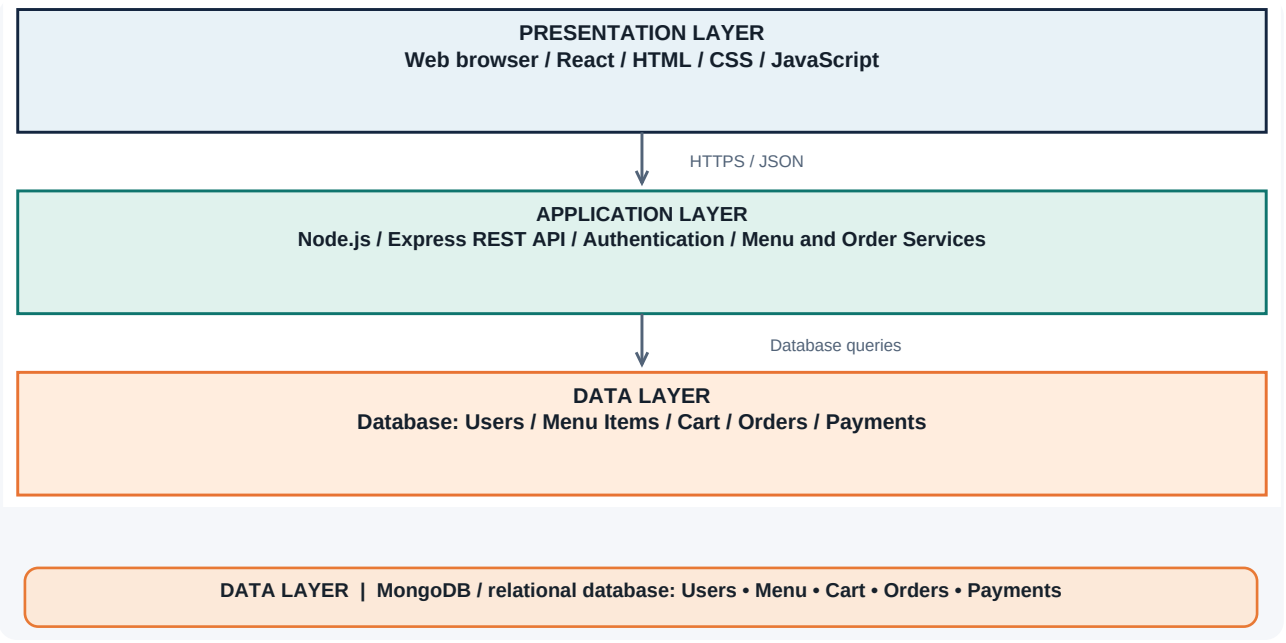


Figure 1: Three-Tier Architecture of the Online Food Ordering System

2.1.1 Software Development Lifecycle (SDLC) Process Model

An Agile incremental model is suitable for this project because the system can be developed and tested in small modules. Each iteration delivers a working feature and incorporates feedback before the next module is started.

Model	Core Philosophy	Suitability
Waterfall	Sequential requirements, design, coding and testing.	Easy to plan, but changes discovered late are expensive.
Spiral	Repeated cycles with explicit risk analysis.	Useful for high-risk projects but may be excessive for a small academic prototype.
Agile Incremental	Builds and tests small working features in iterations.	Selected: authentication, menu, cart, ordering and admin modules can be delivered and reviewed separately.

Table 2: Comparison of SDLC Models

Suggested iterations: Sprint 1 – login and registration; Sprint 2 – menu and cart; Sprint 3 – checkout and order placement; Sprint 4 – admin order management, testing and integration.

2.1.2 Data Flow Architecture (DFD Models)

DFD Level 0 – Context Diagram: The complete application is represented as one process. Customers submit registration details, menu requests and orders, and receive menu information, bills and order-status updates. Administrators maintain menu data and process orders. Delivery personnel receive assigned delivery information and return delivery-status updates.

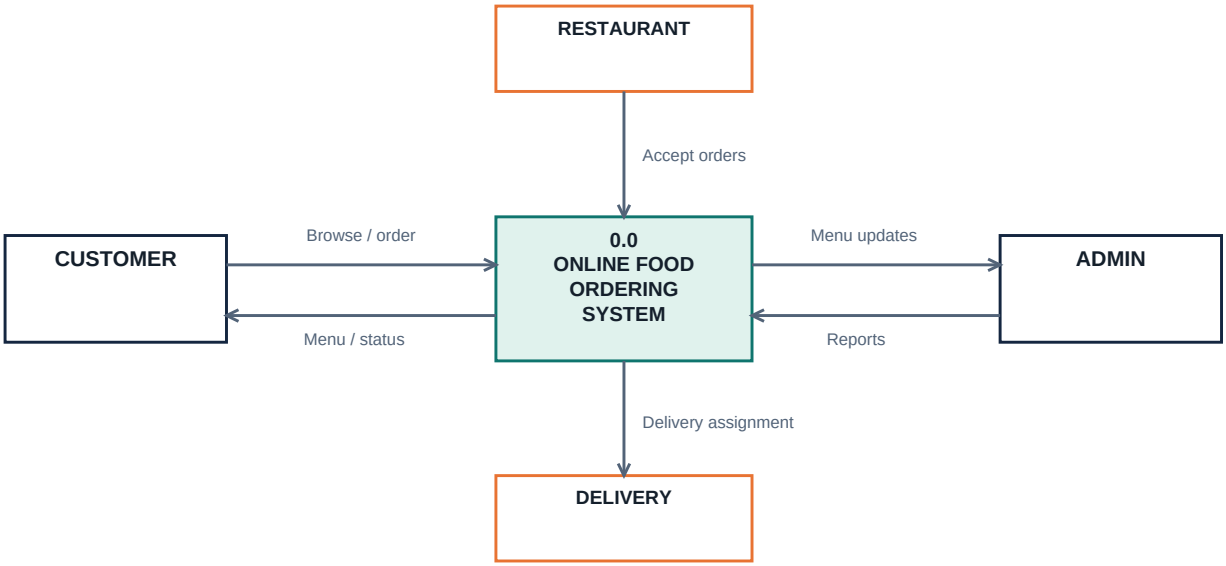


Figure 2: DFD Level 0 – Context Diagram

DFD Level 1 – Main System Processes: The system receives customer and administrator inputs, processes authentication, menu operations, cart and order requests, and stores the resulting information in the database.

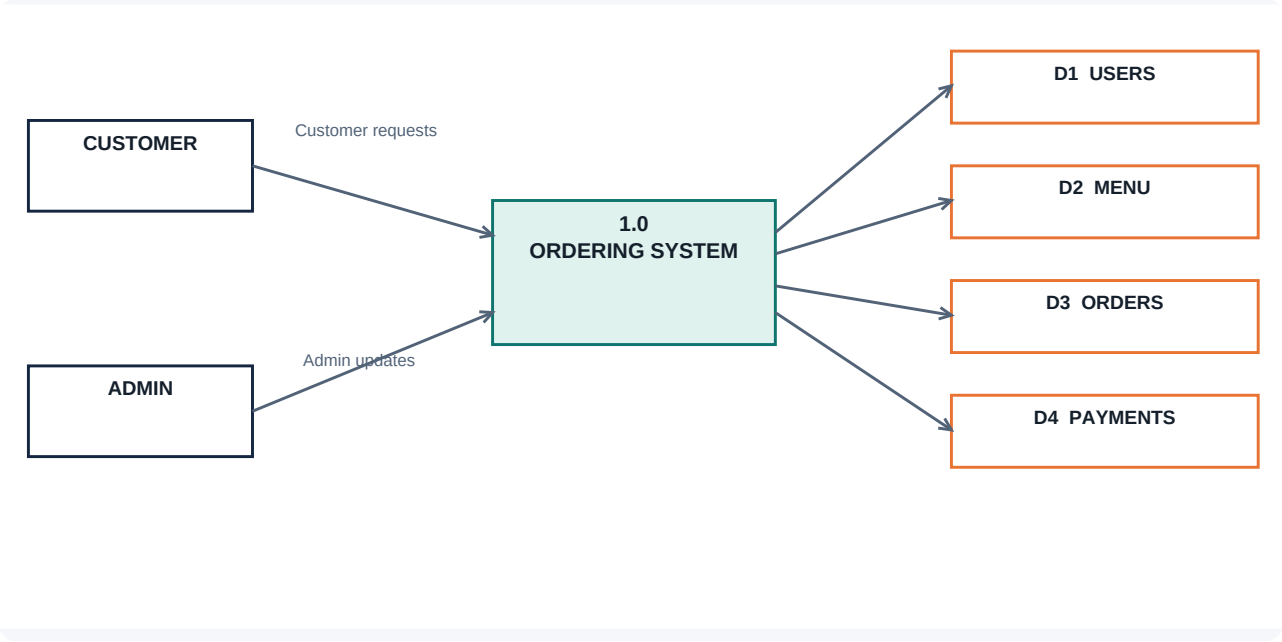


Figure 3: DFD Level 1 – Main Ordering Process

DFD Level 2 – Order Processing Decomposition: The order workflow is decomposed into authentication, menu/cart handling, order placement and payment/status processing. These processes interact with user, menu, order and payment data stores.

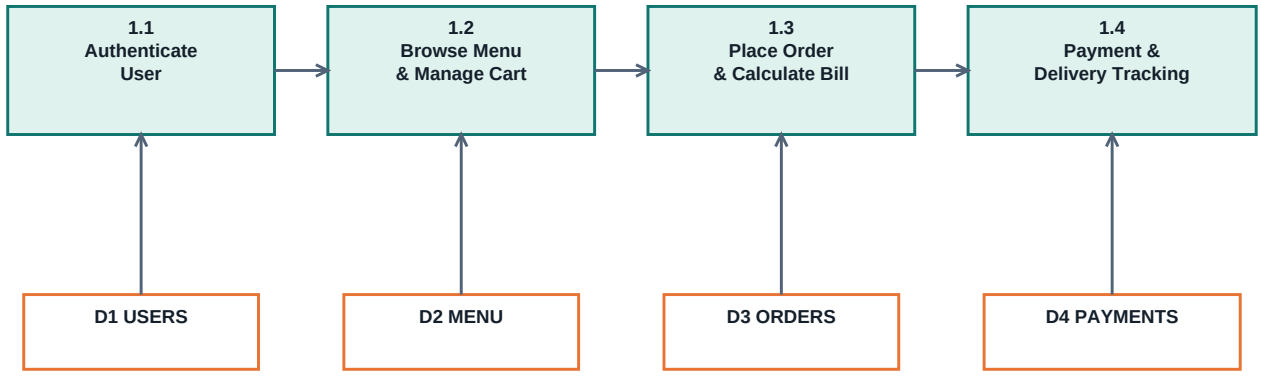


Figure 4: DFD Level 2 – Order Processing Sub-processes

2.1.3 Data Model and Entity-Relationship Architecture

The database stores customer information, menu items, orders, individual order items, payment records and delivery addresses. A relational implementation may use primary and foreign keys; a MongoDB implementation may represent related records through document references or embedded order-item snapshots.

- User: userId, name, email, passwordHash, phone, role.
- MenuItem: itemId, name, description, category, price, imageUrl, availability.
- Order: orderId, userId, addressId, orderDate, totalAmount, orderStatus.
- OrderItem: orderItemId, orderId, itemId, quantity, unitPrice, subtotal.
- Payment: paymentId, orderId, paymentMethod, paymentStatus, transactionReference.
- Address: addressId, userId, recipientName, phone, street, city, postalCode.

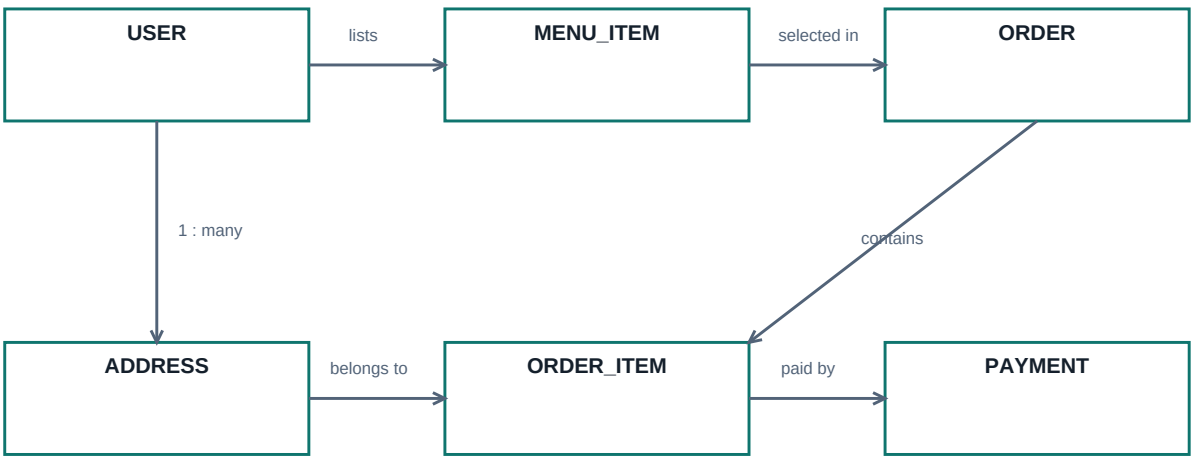


Figure 5: Conceptual Entity-Relationship Diagram

2.2 Product Functions

The main actors and their responsibilities are summarised below. The exact availability of delivery-personnel functions depends on whether a separate delivery role is included in the prototype.

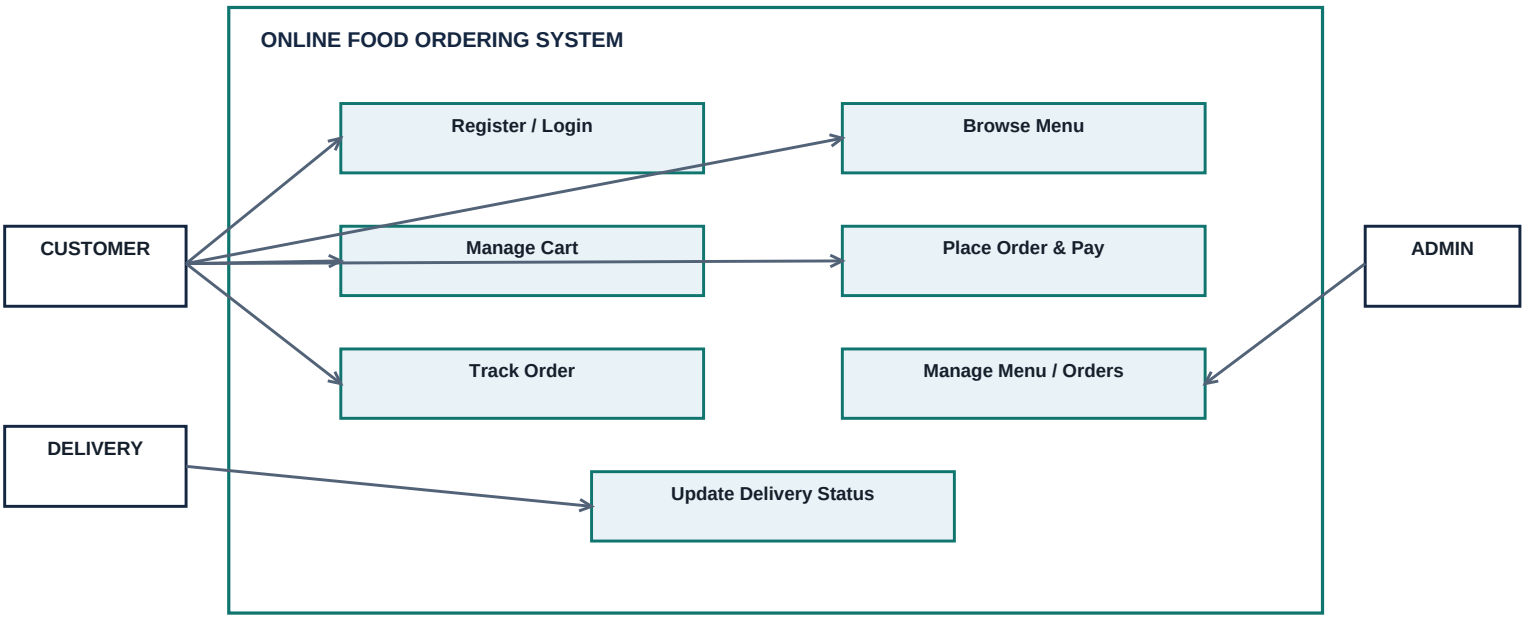


Figure 6: UML-Style Use Case Overview

Actor	Primary Use Cases	Operational Scope
Customer	Register/login, browse menu, manage cart, place order, pay, track order, view history.	Selects food, confirms delivery information and receives order updates.
Administrator	Manage menu, update availability, view orders, change order status, view basic reports.	Controls food catalogue and coordinates order preparation.
Delivery Personnel	View assigned orders, update pickup and delivery status.	Completes delivery and updates the order progress.

Table 3: Actor-to-Use-Case Mapping

2.3 User Classes and Characteristics

- Customers (High frequency, varied technical skill): need a simple, responsive interface and clear order confirmation.
- Restaurant administrators (Medium frequency, moderate technical skill): manage menu details, prices, availability and order status.
- Delivery personnel (Task-based use): need a minimal interface to view assigned deliveries and update status.

2.4 Operating Environment

- Client: modern browsers such as Chrome, Firefox, Edge and Safari on desktop or mobile.
- Frontend: HTML5, CSS3 and JavaScript; React may be used for a component-based interface.
- Backend: Node.js runtime with Express.js REST APIs.
- Database: MongoDB or a relational database such as MySQL/PostgreSQL.
- Network: internet or local network connectivity using HTTP during development and HTTPS in deployment.

2.5 Design and Implementation Constraints

- The application must validate user input on both client and server sides.
- Order totals must be recalculated by the server; client-submitted totals must not be trusted.

- An order must retain the item name and unit price at the time of purchase for accurate historical bills.
- The interface must adapt to mobile, tablet and desktop screen sizes.
- Payment gateway credentials and database secrets must not be hard-coded in source files.

2.6 User Documentation

- Customer guide: account creation, menu browsing, cart management, checkout and order tracking.
- Administrator guide: adding/editing menu items, changing availability and processing orders.
- Developer guide: project setup, database configuration, API endpoints and test instructions.

2.7 Assumptions and Dependencies

- The restaurant maintains accurate menu descriptions, prices and item availability.
- Customers provide a valid delivery address and reachable contact number.
- Online payment depends on the availability and configuration of an external payment gateway.
- Order notifications depend on network connectivity; the prototype may display status inside the application only.

3 External Interface Requirements

3.1 User Interfaces

The frontend should provide a clean and responsive interface with clear navigation and readable food information. The initial prototype requires one representative login page; other screens are described textually and do not require additional screenshots in this report.

- Login page: email/phone field, password field, sign-in button, registration link and validation messages.
- Customer menu: category navigation, food-item cards, price, availability and add-to-cart control.
- Cart and checkout: selected items, quantity controls, subtotal, delivery fee, total and address confirmation.
- Order tracking: order number, current status and timestamped status history.
- Administrator panel: menu-item form, availability toggle and incoming-order list.

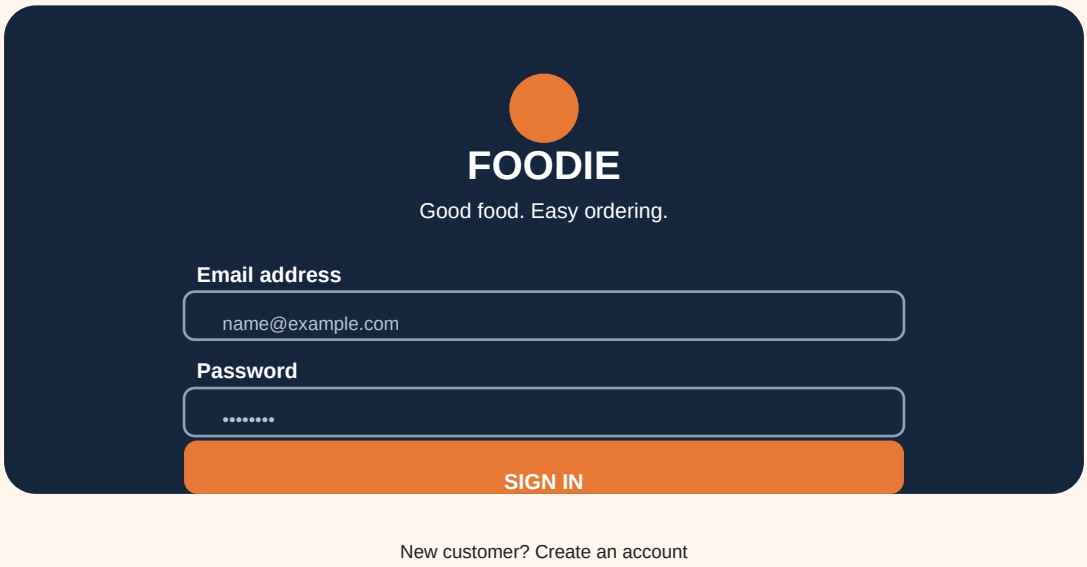


Figure 7: Login Page Mockup for the Online Food Ordering System

3.2 Hardware Interfaces

No specialised hardware is required. Users access the application through a smartphone, tablet or computer with a web browser and network connectivity. A restaurant may use an ordinary computer or tablet to access the administrator panel.

3.3 Software Interfaces

- Frontend communicates with backend endpoints using JSON over HTTP/HTTPS.
- Backend communicates with the database through a suitable driver or ORM/ODM.
- Optional payment gateway interface handles payment initiation and verified payment callbacks.
- Optional email/SMS service may send order confirmation and delivery updates.

3.4 Communications Interfaces

- HTTPS should be used for production communication to protect credentials and personal data.
- REST-style API requests use JSON request and response bodies.
- The server must validate request origin and configure CORS only for trusted frontend origins.
- Payment status must be verified using trusted server-side gateway responses, not only browser redirects.

4 System Features

4.1 User Registration and Authentication

Allows customers and administrators to access the system using verified credentials. Priority: High.

Stimulus/Response Sequence

- Stimulus: user enters email/phone and password on the login form.
- Response: server validates credentials and starts an authenticated session or returns an error.

Functional Requirements

- FR-1.1: The system shall allow a new customer to register with name, email, phone and password.
- FR-1.2: The system shall store passwords as secure hashes and shall never store plaintext passwords.
- FR-1.3: The system shall reject invalid credentials and display a clear error message.
- FR-1.4: The system shall restrict administrator functions to authorised administrator accounts.

4.2 Food Menu Browsing and Search

Displays available food items with relevant details. Priority: High.

Stimulus/Response Sequence

- Stimulus: customer opens the menu or searches for a food item.
- Response: system displays matching available items and their prices.

Functional Requirements

- FR-2.1: The system shall display item name, description, category and price.
- FR-2.2: The system shall allow filtering by category and searching by item name.
- FR-2.3: Unavailable items shall be clearly marked and shall not be added to the cart.

4.3 Shopping Cart Management

Allows customers to select food items and adjust quantities before checkout. Priority: High.

Stimulus/Response Sequence

- Stimulus: customer adds an item or changes its quantity.
- Response: cart contents and subtotal are recalculated.

Functional Requirements

- FR-3.1: The system shall allow customers to add and remove menu items from the cart.
- FR-3.2: The system shall allow quantity updates using positive integer values.
- FR-3.3: The system shall calculate each line subtotal as quantity multiplied by unit price.
- FR-3.4: The system shall preserve cart data during the active session.

4.4 Order Placement and Bill Calculation

Converts a confirmed cart into an order. Priority: High.

Stimulus/Response Sequence

- Stimulus: customer confirms the cart, delivery address and payment method.
- Response: system validates availability, calculates the final amount and creates an order.

Functional Requirements

- FR-4.1: The system shall require a non-empty cart and a valid delivery address.

- FR-4.2: The server shall calculate subtotal, delivery fee, applicable tax/discount and final total.
- FR-4.3: The system shall generate a unique order identifier.
- FR-4.4: The system shall store an order-item snapshot including item name, quantity and purchase price.

4.5 Payment Processing

Supports configured payment options. Priority: High for online payment; Medium for cash on delivery.

Stimulus/Response Sequence

- Stimulus: customer selects a payment method and confirms checkout.
- Response: system records the payment attempt and updates payment status after verification.

Functional Requirements

- FR-5.1: The system shall support cash on delivery where enabled.
- FR-5.2: If an online gateway is configured, the system shall redirect or invoke its secure payment flow.
- FR-5.3: The system shall mark online payment successful only after trusted server-side verification.
- FR-5.4: Failed or cancelled payments shall be recorded without falsely marking the order as paid.

4.6 Order Tracking and Status Updates

Provides visibility into order progress. Priority: High.

Stimulus/Response Sequence

- Stimulus: customer opens order details.
- Response: system displays the latest order status and status history.

Functional Requirements

- FR-6.1: The system shall support statuses such as Placed, Confirmed, Preparing, Ready for Pickup, Out for Delivery, Delivered and Cancelled.
- FR-6.2: Authorised staff shall be able to update order status.
- FR-6.3: Customers shall be able to view the latest status for their own orders only.

4.7 Administrator Menu and Order Management

Provides operational control to restaurant staff. Priority: High.

Stimulus/Response Sequence

- Stimulus: administrator adds a menu item or opens an incoming order.
- Response: system validates and saves the update or displays order details.

Functional Requirements

- FR-7.1: Administrators shall be able to create, edit and deactivate menu items.
- FR-7.2: Administrators shall be able to update price, description, category and availability.
- FR-7.3: Administrators shall be able to view incoming orders and update their status.
- FR-7.4: The system shall record the date and time of important order-status changes.

4.8 Order History and Customer Profile

Allows customers to review previous orders and maintain contact details. Priority: Medium.

Stimulus/Response Sequence

- Stimulus: customer opens profile or order history.
- Response: system displays saved profile information and previous orders.

Functional Requirements

- FR-8.1: Customers shall be able to view their own order history.
- FR-8.2: Customers shall be able to update permitted profile and delivery-address fields.
- FR-8.3: The system shall prevent one customer from viewing another customer's private order details.

5 Other Nonfunctional Requirements

5.1 Performance Requirements

- Common menu and order-history requests should respond within approximately 2 seconds under normal academic prototype load.
- The interface should show a visible loading state when an operation takes longer than expected.
- The system should paginate or limit long menu and order-history results where necessary.

5.2 Safety and Reliability Requirements

- If order creation fails, the system shall not display a false success confirmation.
- The system shall avoid duplicate orders when a customer accidentally submits checkout more than once.
- Database backups should be maintained according to the deployment environment.
- Payment and order status updates must be handled consistently to avoid mismatched records.

5.3 Security Requirements

- Passwords must be hashed using a recognised password-hashing algorithm.
- Authenticated endpoints must verify the user's identity and role.
- Server-side validation and parameterised database operations must be used to reduce injection risks.
- Sensitive data and payment credentials must be protected in transit and must not be exposed in logs.
- Customers may access only their own profile, address and order records.

5.4 Software Quality Attributes

- Usability: ordering steps should be clear and require minimal navigation.
- Reliability: the system should preserve confirmed orders and display accurate statuses.
- Maintainability: frontend, backend and data-access responsibilities should be separated.
- Portability: the application should run in modern browsers and on common operating systems.
- Accessibility: controls should have readable labels, sufficient contrast and keyboard support.

5.5 Business Rules

- BR-1: A customer must be authenticated before placing an order.
- BR-2: Only items marked available may be added to a new cart or confirmed order.
- BR-3: The server is the authoritative source for item prices, delivery fees and order totals.
- BR-4: An order cannot be marked Delivered before it has been dispatched or marked Out for Delivery.
- BR-5: Only authorised administrators or assigned delivery personnel may update operational order statuses.
- BR-6: A customer may cancel an order only while its status is within the restaurant's permitted cancellation window.

6 Other Requirements

6.1 Frontend Design and Interface Description

The proposed frontend is a responsive web interface. This report includes one representative login-page mockup only, as requested. The remaining screens are specified through functional descriptions rather than multiple frontend screenshots.

6.1.1 Customer Login Portal

Figure 7 illustrates a simple login page branded as FOODIE. It contains an email field, password field, sign-in button and a link for new customers to create an account. The implementation should validate empty fields, display meaningful error messages and provide a secure password-handling flow.

6.1.2 Menu and Cart Interface

After successful login, the customer can browse menu categories and food cards. Each card should show the food name, short description, price and availability. The Add to Cart action updates the cart summary. The cart allows quantity changes and item removal before checkout.

6.1.3 Checkout and Order Confirmation

The checkout page displays the selected delivery address, ordered items, quantities, subtotal, delivery charge and final amount. Before creating the order, the system revalidates current item availability and prices. A confirmation screen displays the generated order number and payment status.

6.1.4 Administrator Interface

The administrator interface provides forms for adding and editing menu items and a list of incoming orders. Each order displays its identifier, customer details required for fulfilment, ordered items, payment status and current order status.

6.2 Backend Architecture and Operational Mechanics

6.2.1 Runtime Architecture

The backend may be organised into routes, controllers, services and database-access modules. Routes expose endpoints, controllers validate requests, service functions implement business rules, and the data layer reads or writes persistent records.

Module	Example Responsibility
Authentication	Register, login, logout and role verification.
Menu	List, search, create, update and deactivate menu items.
Cart / Order	Validate cart, calculate bill and create orders.
Payment	Initiate payment and verify payment result.
Delivery / Status	Assign delivery and maintain order-status history.
Administration	Manage users, menu items and operational reports.

6.2.2 Authentication and Role-Based Access

On registration, the backend validates the submitted fields and stores a password hash. During login, the backend verifies the password and issues a secure session or signed token. Protected routes check authentication and role permissions before allowing access to administrative or delivery operations.

6.2.3 Order Creation and Transaction Workflow

The order service should execute the following logical sequence:

- Validate authenticated customer and delivery address.
- Retrieve current menu-item records and verify availability.

- Recalculate item subtotals and final bill on the server.
- Create the order and its order-item records consistently.
- Record payment method and initial payment status.
- Return the order identifier and confirmation to the customer.

6.2.4 Order Status Workflow

The order status follows a controlled progression: Placed → Confirmed → Preparing → Ready for Pickup → Out for Delivery → Delivered. Cancellation may be allowed from selected earlier states according to restaurant policy. Every status change should include a timestamp and the authorised actor.

6.3 Database and Data Persistence Requirements

- User records must have unique identifiers and unique email addresses where email is used for login.
- Each order must be linked to one customer and contain one or more order-item records.
- Payment records must be linked to the corresponding order.
- Historical order items must preserve the purchase-time name and unit price.
- Database constraints or application validation must prevent invalid quantities and orphan records.

6.4 Privacy and Compliance Considerations

The application should collect only information needed to authenticate users, fulfil orders and process payments. Customer addresses and contact details must be visible only to authorised users involved in order fulfilment. A production deployment must follow applicable privacy, consumer-protection and payment-security requirements.

A Glossary

- API: Application Programming Interface; a defined way for software components to communicate.
- Authentication: Verification of a user's identity.
- Cart: Temporary collection of food items selected before checkout.
- CRUD: Create, Read, Update and Delete operations on stored data.
- DFD: Data Flow Diagram; a model showing movement of data between entities, processes and data stores.
- ERD: Entity-Relationship Diagram; a model showing entities and their relationships.
- FR: Functional Requirement; a statement describing a required system behaviour.
- JWT: JSON Web Token; a signed token format commonly used to represent authenticated claims.
- RBAC: Role-Based Access Control; access permissions based on a user's role.
- REST: Representational State Transfer; an architectural style commonly used for web APIs.
- SRS: Software Requirements Specification; a document describing system requirements.
- UI: User Interface; the visible part through which a user interacts with the application.

B Analysis Models

The principal analysis models are included in Section 2: the three-tier architecture (Figure 1), DFD Level 0 (Figure 2), DFD Level 1 (Figure 3), DFD Level 2 (Figure 4), conceptual ER diagram (Figure 5) and use-case overview (Figure 6). The login-page mockup is presented in Section 3.1 (Figure 7). These models describe the proposed system at a conceptual level and should be refined during implementation.

C To Be Determined (TBD) List

Item ID	Description	Responsible Entity	Status
TBD-01	Final database choice and deployment configuration.	Project Developer	To be decided
TBD-02	Payment gateway selection and credentials, if online payment is enabled.	Project Developer / Restaurant	To be decided
TBD-03	SMS or email notification provider, if notifications are required.	Project Developer	Optional
TBD-04	Delivery assignment method and cancellation policy.	Restaurant Administrator	To be confirmed

Table 4: To Be Determined Items

— End of Report —